

Python Engineer – Test Assignment

We are excited to see your skills in action! Below are three tasks for you to choose from. Please pick one or more tasks to complete. The key requirements are clean, efficient code and thorough documentation.

Task 1: Drone Path Prediction

Objective: Using a video file ([video1](#)), extract coordinates and construct a drone's flight path on a map.

Steps:

1. Apply an **Optical Flow** algorithm (using OpenCV or any other library) to the video to track the motion of points.
2. Based on the motion data, predict the drone's path.
3. Plot the path on a map for visualization.
4. (Optional) Implement the task using alternative motion estimation algorithms for comparison (e.g., Deep Learning approaches).

Deliverables:

- Code to process the video and predict the drone's path.
 - A map visualization showing the predicted path.
 - Documentation explaining the approach, tools used, and results.
-

Task 2: Signal Bot with ATAK Integration

Objective: Create a Signal bot to send geolocation and target information to an ATAK (Android Team Awareness Kit) client.

Steps:

1. Design a Python-based Signal bot that can send messages like:
"48.567123 39.87897 tank" (longitude, latitude, target description).
2. Format the message in **Cursor on Target (CoT)** protocol.
3. Send the formatted CoT message to an ATAK client (<https://www.civtak.org/>).
4. Ensure that the message is displayed in the ATAK client interface.

Deliverables:

- Python script for the Signal bot.
 - Documentation on how to set up and run the bot, as well as a description of the CoT protocol formatting.
 - Confirmation or demonstration that the messages display correctly in ATAK.
-

Task 3: Car Path Tracking

Objective: Track moving cars from a video file ([video2](#) + [srt2](#)) and plot their paths on a map.

Steps:

1. Analyze the video to detect and track moving cars.
 - You can use methods like background subtraction, object detection models, or Optical Flow.
2. Extract the paths of the detected cars.
3. Plot the paths on a map for visualization.

Deliverables:

- Code to detect cars, track their paths, and plot them on a map.
 - A map visualization showing the paths of all detected cars.
 - Documentation detailing your approach, methods, and any assumptions made.
-

Task 4: VIPE Gaussian Splatting Demo

Objective: Run the [VIPE](#) project locally or on a GPU cloud instance (e.g. RunPod.io), generate a Gaussian Splatting reconstruction from a provided dataset, record a short demo video, and prepare a public GitHub repository with instructions.

Input

- [VIPE](#)
- [Dataset](#)

Steps

1. Set up the environment and install all dependencies required to run VIPE
 - Either on your local machine (Linux, NVIDIA GPU) or on RunPod.io
2. Prepare the dataset in the format required by VIPE (if preprocessing is needed, script it).
3. Run VIPE to obtain a Gaussian Splatting representation of the scene.
4. Render a short trajectory / camera path using the resulting Gaussian Splatting model.

Deliverables

- Working VIPE setup (local or RunPod)
 - Gaussian Splatting result generated from the dataset
 - Public GitHub repo with instructions and scripts
 - Demo video demonstrating the full pipeline end-to-end
-

Submission Guidelines

1. **Code:**
 - Ensure clean, well-structured, and modular code.
 - Include comments and docstrings to explain key parts of your implementation.
2. **Documentation:**
 - A brief overview of your approach, tools used, and any challenges encountered.
 - Instructions on how to run the code.
3. **Submission:**
 - Share your work as a GitHub repository or a zip file with a clear directory structure.

We look forward to reviewing your solutions. Good luck, and don't hesitate to showcase your creativity and technical skills!